



IgrejasWeb System OS

Roadmap Oficial de Desenvolvimento

Versão 2.0

Junho 2026

17 Fases

Confidencial

Progresso Geral — Fase 1 de 17 · 18% concluído

7 entregas concluídas · 1 em andamento · 16 pendentes

STATUS DE DESENVOLVIMENTO — ATUALIZADO EM 22/06/2026

FASE 0 — FUNDAÇÃO (CONCLUÍDA)

Migration 001 — Schema Base

Migration 002 — RLS Policies

Migration 003 — Seed (2 campos)

Auth: Login → Contexto →
Dashboard

Layout: Sidebar + Header +
Breadcrumb

Dashboard Home: StatCards +
Módulos

FASE 1 — ESTRUTURA ADMINISTRATIVA

1A — Gestão de Unidades
(validada)

1B — Gestão de Usuários (validada)

☒ Migration 004 — Usuários Teste
N0–N4

☒ 1C — Configuração de Módulos

FASES 2–10 — MÓDULOS OPERACIONAIS PRINCIPAIS

☐ Fase 2 — Membros (NÚCLEO)

☐ Fase 3 — Secretaria

☐ Fase 4 — EBD

☐ Fase 5 — Escola Teológica

☐ Fase 6 — Cursos Livres

☐ Fase 7 — Financeiro

☐ Fase 8 — Eventos

☐ Fase 9 — Patrimônio

☐ Fase 10 — Ocorrências

FASES 11–17 — MÓDULOS ENTERPRISE AVANÇADOS

☐ Fase 11 — Comunicação
Omnichannel

☐ Fase 12 — Voluntários & Escalas

☐ Fase 13 — Ministério Infantil

☐ Fase 14 — Governança &
Compliance

☐ Fase 15 — Analytics & BI

☐ Fase 16 — API Pública &
Integrações

☐ Fase 17 — PWA & App Mobile

1. Visão Geral do Projeto

O **IgrejasWeb System OS** é um sistema de gestão eclesiástica multi-tenant de nível Enterprise, projetado para operar múltiplos campos/ministérios de forma isolada e segura. Cada ministério possui dados, usuários, módulos e configurações independentes. O sistema é construído em camadas, da fundação de segurança até os módulos operacionais e, por fim, inteligência analítica.



VISÃO ENTERPRISE

Este roadmap foi ampliado para 17 fases, incorporando todos os módulos de um ChMS Enterprise (Church Management System) completo: multi-campus, financeiro com Fund Accounting, comunicação omnichannel, ministério infantil, governança, BI e API pública. O sistema é arquiteturalmente compatível com white-label para outros segmentos.

Stack Tecnológica

Framework	Next.js 15.3 — App Router + Turbopack + React 19
Linguagem	TypeScript 5 (strict mode em todo o projeto)
Banco de Dados	PostgreSQL via Supabase — RLS ativo, funções PL/pgSQL, triggers
Autenticação	Supabase Auth + cookie httpOnly iw_context (JWT-like SSR)
Storage	Supabase Storage (fotos, documentos, certificados, ativos)
Estilos	CSS Custom Properties (design system próprio) + Tailwind v4
Deploy	Vercel (frontend + Edge Functions) + Supabase Cloud
Mobile (futuro)	React Native + Expo (compartilha lógica TypeScript)
Analytics (futuro)	Metabase ou Recharts com views materializadas no Postgres

Princípios Arquiteturais Fundamentais

- › **Multi-tenant por ministry_id:** todas as tabelas possuem ministry_id NOT NULL com RLS ativo — dados de um ministério nunca são acessíveis por outro, em nenhuma

camada

- › **Security by Design / Shift-Left:** RLS implementado antes de qualquer módulo operacional. Jamais retrofitado
- › **Party Pattern:** toda pessoa (membro, aluno, fornecedor, voluntário) é uma Party única com CPF. Papéis são atribuídos via party_roles — sem duplicação de cadastro
- › **Feature Flags por Ministério:** cada módulo pode ser ativado ou desativado via ministry_modules sem alterar código — comportamento diferente por campo sem deploys
- › **RBAC 5 Níveis:** N0 Super Master → N1 Admin Campo → N2 Admin Sede → N3 Admin Setor → N4 Usuário Local — com guard em Server Action e em UI
- › **Hierarquia de Unidades:** CAMPO → SEDE → SETOR → IGREJA → CÉLULA → PONTO_PREGAÇÃO — com is_headquarters e is_sector_mother para regras de negócio
- › **Server Actions como API interna:** toda mutação passa por Server Action com assertCanManage(ctx) — nunca acesso direto ao banco pelo client
- › **White-label Ready:** a arquitetura é agnóstica ao domínio eclesiástico. O mesmo engine de RBAC/multi-tenant serve franquias, clínicas, redes de ensino

2. Fundação Concluída ✓ 100% PRONTO

Banco de Dados (Migrations 001-004)

- ✓ **001_base_schema.sql** — 6 tabelas base: ministries, ministry_modules, units, parties, party_roles, admin_roles. Índices em ministry_id, parent_id, user_id
- ✓ **002_rls_policies.sql** — RLS em todas as tabelas com políticas por nível. Funções helper: is_super_master(), get_user_level(), get_user_ministry_id()
- ✓ **003_seed.sql** — 2 ministérios reais (Piracicaba + São Paulo), 15 setores com igrejas, 9 módulos configurados por campo
- ✓ **004_test_users.sql** — 7 usuários de teste N0-N4 com bcrypt (crypt/gen_salt), auth.identities correto, admin_roles vinculados

Autenticação e Contexto de Sessão

- ✓ **/login** — Formulário com DEV dropdown de 7 usuários organizados por nível (N0-N4) com auto-preenchimento de senha
- ✓ **/contexto** — Seleção de ministério para N0 (Super Master vê todos os campos) + geração do cookie iw_context
- ✓ **iw_context cookie** — Contém: user_id, ministry_id, ministry_name, unit_id, level, modules[] — disponível em todo Server Component via cookies()
- ✓ **middleware.ts** — Proteção de rotas: /dashboard requer cookie válido, /admin requer level ≤ 1

Layout e Dashboard

- ✓ **Sidebar colapsável** — Módulos dinâmicos (filtrados por ctx.modules[]), seção Admin visível só para N0/N1, ícones SVG inline, animação de colapso
- ✓ **Header** — Breadcrumb inteligente com match por prefixo, badge de nível RBAC colorido por nível, nome do ministério
- ✓ **Dashboard Home** — StatCards (membros, unidades, módulos, usuários), grid de módulos ativos com link, banner do campo

Fase 1A — Gestão de Unidades VALIDADA

- ✓ Árvore hierárquica recursiva com collapse automático em profundidade ≥ 2
- ✓ CRUD completo: criar, editar, toggle ativo/inativo por unidade
- ✓ Formulário com seleção de tipo (CAMPO/SEDE/SETOR/IGREJA/CÉLULA), parent_id inteligente (filtra pais válidos por tipo), is_headquarters, is_sector_mother

- ✓ Busca em tempo real + filtro por tipo + toggle "mostrar inativas"
- ✓ Stats bar: Total / Ativas / Inativas / Exibindo
- ✓ Guarda de permissão: N2+ redireciona para /dashboard

Fase 1B — Gestão de Usuários **VALIDADA**

- ✓ Listagem de todos os usuários do ministério com e-mail, nível, unidade vinculada e status
- ✓ Convite por e-mail via Supabase Admin API (inviteUserByEmail) com definição de nível e unidade no ato do convite
- ✓ Edição de nível e unidade (respeitando: só pode editar quem tem nível menor)
- ✓ Revogação (is_active = false) e reativação de acesso com confirmação
- ✓ Painel DEV em collapsible <details> com tabela de todos os usuários de teste
- ✓ Filtro de unidades no modal: N1→CAMPO, N2→SEDE, N3→SETOR, N4→IGREJA/CÉLULA

3. Visão Enterprise — Gap Analysis

Mapeamento dos 15 módulos Enterprise ChMS vs. estado atual do projeto. A coluna "Fase" indica quando cada módulo será construído neste roadmap.

#	MÓDULO ENTERPRISE	STATUS	OBSERVAÇÃO	FASE
1	Multi-filial / Multi-campus	☒	Fundação completa: ministry_id, RBAC 5 níveis, RLS, seed com 2 campos	Fase 0
2	Gestão de Membros	☐	Party Pattern modelado, tabelas member_history e member_transfers planejadas	Fase 2
3	Financeiro e Contábil	☐	Tabelas fin_* planejadas. Inclui Fund Accounting, orçamento vs. realizado, relatórios por unidade	Fase 7
4	Governança e Compliance	☐	Atas de reunião, mandatos, votações eletrônicas, audit trail, LGPD	Fase 14
5	Comunicação Omnichannel	☐	E-mail (SendGrid), WhatsApp Business API, Push. Requer workers assíncronos (Edge Functions)	Fase 11
6	Eventos e Agenda	☐	Calendário, inscrições com controle de vagas, check-in, relatório de participação	Fase 8
7	Voluntários e RH	☐	Cadastro, escalas automáticas, confirmação via app, background check futuro	Fase 12
8	Ministério Infantil	☐	Check-in/check-out seguro, QR Code, vinculação responsável→criança, alertas de emergência	Fase 13
9	Analytics e BI	☐	Dashboards executivos, KPIs, exportação Power BI. Requer views materializadas no Postgres	Fase 15
10	App Mobile	☐	Next.js como PWA cobre 70% dos casos. App nativo (React Native + Expo) para check-in infantil e offline	Fase 17

#	MÓDULO ENTERPRISE	STATUS	OBSERVAÇÃO	FASE
11	Patrimônio e Facilities	☐	Inventário por unidade, depreciação automática, movimentação entre igrejas	Fase 9
12	Segurança Enterprise		RBAC  , RLS  . Pendente: MFA, SSO (Google Workspace / Azure AD), LGPD completo	Fase 14
13	Integrações e API REST	☐	Next.js Route Handlers são a API. Faltam: documentação OpenAPI, rate limiting, webhooks, API keys	Fase 16
14	Conteúdo e Streaming	☐	Arquivo de sermões, biblioteca EBD, plataforma de cursos online — parcialmente coberto pelas Fases 5 e 6	Fase 6 / 17
15	Secretaria e Documentos	☐	Templates HTML editáveis, emissão de PDF automático por membro, histórico de documentos	Fase 3

4. Mapa de Dependências e Ordem Cronológica

A ordem abaixo é técnica e obrigatória. Um módulo só pode ser iniciado após seus pré-requisitos estarem completamente validados em ambiente de desenvolvimento.

Fase	Módulo	Depende de	Novas tabelas no banco	Esforço
0	Fundação FEITA	—	ministries, ministry_modules, units, parties, party_roles, admin_roles	—
1A	Gestão de Unidades FEITA	Fundação	— (usa units)	—
1B	Gestão de Usuários FEITA	1A	— (usa admin_roles)	—
1C	Config de Módulos EM ANDAMENTO	1B	— (usa ministry_modules)	1-2 dias
2	Membros NÚCLEO	Fase 1 completa	member_history, member_transfers	2-3 sem.
3	Secretaria	Membros	document_types, documents	1 sem.
4	EBD	Membros + Unidades	ebd_classes, ebd_roll_calls, ebd_attendances	1 sem.
5	Escola Teológica	Membros (role ALUNO)	school_semesters, school_disciplines, school_enrollments, school_grades, school_attendance	2 sem.
6	Cursos Livres	Membros	courses, course_enrollments, course_certificates	1 sem.
7	Financeiro CRÍTICO	Membros + Unidades	fin_accounts, fin_categories, fin_transactions, fin_transfers, fin_budgets	3-4 sem.

Fase	Módulo	Depende de	Novas tabelas no banco	Esforço
8	Eventos	Membros + Unidades	events, event_registrations, event_checkins	1 sem.
9	Patrimônio	Unidades	asset_categories, assets, asset_movements	1 sem.
10	Ocorrências SENSÍVEL	Membros + N2+	occurrence_types, occurrences, occurrence_comments	1 sem.
11	Comunicação ENTERPRISE	Membros + Eventos	comm_templates, comm_campaigns, comm_logs	2–3 sem.
12	Voluntários & Escalas ENTERPRISE	Membros + Eventos	volunteer_profiles, ministry_teams, service_schedules, schedule_confirmations	2 sem.
13	Ministério Infantil ENTERPRISE	Membros + Voluntários	child_profiles, child_checkins, child_classes, child_responsibles	2 sem.
14	Governança & LGPD ENTERPRISE	Fase 1 + Membros	governance_meetings, governance_minutes, mandates, votes, lgpd_consent, audit_logs	3 sem.
15	Analytics & BI ENTERPRISE	Todos os módulos	Views materializadas + tabelas de métricas	2–3 sem.
16	API Pública & Integrações ENTERPRISE	Financeiro + Membros	api_clients, api_keys, webhook_endpoints, webhook_logs	2 sem.
17	PWA & App Mobile FUTURO	Fase 1–10 estáveis	push_subscriptions, offline_sync_queue	4–6 sem.

5. Fases Detalhadas

FASE 1 Estrutura Administrativa

 1A — Unidades

 1B — Usuários

 1C — Módulos

1A — Gestão de Unidades CONCLUÍDA

- › Árvore recursiva UnitTree.tsx com collapse automático por profundidade, busca em tempo real e filtro por tipo
- › VALID_PARENT_TYPES: regras de quais tipos podem ser pais de quais — CAMPO só abaixo de NULL, Igreja abaixo de SETOR, etc.
- › is_headquarters e is_sector_mother editáveis no formulário — critérios que afetam lógica de RLS futura
- › Server Actions: criarUnidadeAction, atualizarUnidadeAction, toggleUnidadeAtivaAction — todas com assertCanManage(ctx) e revalidatePath
- › Guard: ctx.level > 1 → redirect("/dashboard") tanto no Server Component quanto nas actions

✓ CRITÉRIO DE VALIDAÇÃO — APROVADO

- ✓ Criar IGREJA dentro do Setor 01 de Piracicaba → aparece na árvore sem reload
- ✓ Editar nome de unidade existente → persiste imediatamente
- ✓ Toggle inativo → unidade some da lista padrão mas aparece no filtro "mostrar inativas"

1B — Gestão de Usuários CONCLUÍDA

- › Supabase Admin API (service_role): createAdminClient() com autoRefreshToken: false, persistSession: false
- › listarUsuariosAction: combina admin_roles + auth.admin.listUsers() — une e-mails com dados de nível/unidade
- › convidarUsuarioAction: verifica se e-mail existe → inviteUserByEmail se novo, reativa is_active se revogado
- › atualizarUsuarioAction: guard ctx.level >= target.level (não pode editar par ou superior)
- › revogarAcessoAction / reativarAcessoAction: is_active toggle com confirmação no client
- › UserFormModal: radio cards de nível N1–N4 com cor, unidades filtradas por nível selecionado

✓ CRITÉRIO DE VALIDAÇÃO — APROVADO

- ✓ Convidar usuário → e-mail Supabase enviado e registro em admin_roles criado
- ✓ Revogar acesso → is_active = false → login bloqueado
- ✓ Editar nível/unidade → cookie iw_context atualizado no próximo login

1C — Configuração de Módulos por Ministério 🟦 EM ANDAMENTO

- › Tela /dashboard/admin/modulos com listagem de todos os 9 módulos disponíveis
- › Toggle individual por módulo: atualiza ministry_modules.is_active via Server Action
- › Ao salvar, exibir alerta: "Mudança refletida no próximo login dos usuários"
- › N2+ não tem acesso a esta tela (redirect para /dashboard)
- › Exibir descrição e ícone de cada módulo para facilitar decisão do admin

✓ CRITÉRIO DE VALIDAÇÃO

- ✓ Desativar módulo Financeiro para São Paulo → item some do sidebar após novo login
- ✓ Reativar → item retorna ao sidebar
- ✓ Piracicaba não é afetada pela mudança em São Paulo (isolamento)

FASE 2

Módulo Membros — NÚCLEO DO SISTEMA

Dep: Fase 1 validada

2–3 semanas

CORE

O módulo mais crítico de todo o sistema. Quase todos os módulos seguintes referenciam uma Party (pessoa). Erros no modelo de dados aqui geram retrabalho em cascata em todas as fases seguintes. Implementar com cuidado e testar exaustivamente antes de avançar.



ATENÇÃO — PARTY PATTERN

Um membro é uma Party com party_role MEMBRO. A mesma pessoa pode ser MEMBRO + VOLUNTÁRIO + ALUNO (Escola Teológica) com um único CPF. Nunca criar tabelas separadas para cada papel. Toda tela que lista pessoas deve usar parties com JOIN em party_roles.

Migration necessária

member_history

`id` uuid PK • `ministry_id` uuid FK • `party_id` uuid FK
`status_anterior` member_status • `status_novo` member_status
`motivo` text • `alterado_por` uuid FK (auth.users) • `criado_em` timestampz
Audit trail imutável de todas as mudanças de situação eclesial — nunca deletar registros

member_transfers

`id` uuid PK • `ministry_id` uuid FK • `party_id` uuid FK
`unit_origem_id` uuid FK (units) • `unit_destino_id` uuid FK (units)
`tipo` transfer_type (INTRA, INTER, DESLIGAMENTO, RETORNO)
`data_transferencia` date • `aprovado_por` uuid FK • `obs` text
Transferências inter-unidade e inter-ministério. Preserva histórico completo

Funcionalidades por tela

- › **Lista de membros:** busca por nome/CPF/telefone, filtros por status/setor/igreja/gênero/cargo, paginação, exportação CSV
- › **Ficha do membro (Party):** dados pessoais (nome, CPF, RG, nascimento, gênero, estado civil, foto), endereço completo, contatos (celular, e-mail, WhatsApp), filiação (pai, mãe), cônjuge (link para outra Party), filhos (link para outras Parties)
- › **Ficha eclesial:** data de batismo nas águas, data de batismo no Espírito Santo, cargo eclesiástico (MEMBRO, OBREIRO, DIÁCONO, PRESBÍTERO, PASTOR, BISPO), igreja de origem, data de ingresso na AD Madureira
- › **Situação eclesial:** ATIVO, INATIVO, EM_OBSERVAÇÃO, SUSPENSO, DESLIGADO — com alerta visual por status
- › **Histórico de situação:** linha do tempo com data, responsável, status anterior → novo e motivo
- › **Transferência:** formulário de transferência intra-ministério (para outra unidade) ou inter-ministério (carta de transferência em PDF)
- › **Impressão de ficha:** PDF com todos os dados do membro, foto, QR Code com link para ficha digital

Regras de Negócio

- N3 (Admin Setor) só vê e cria membros da sua unidade (`unit_id`) e filhas — RLS por `unit_id` hierárquico
- N4 (Usuário Local) só vê membros da sua IGREJA específica
- Transferência inter-ministério requer aprovação de N2 (Admin Sede) do ministério de origem
- CPF deve ser validado via mod-11 no backend antes de salvar

→ Foto armazenada no Supabase Storage, máximo 5MB, formatos JPG/PNG, redimensionada para 400×400

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Cadastrar membro completo com todos os campos obrigatórios → ficha exibida corretamente
- ✓ Alterar situação de ATIVO para SUSPENSO com motivo → histórico registrado com responsável
- ✓ Login como N3 Setor 01 → não deve ver membros do Setor 02
- ✓ Transferir membro para outra Igreja → unidade atualizada no party_roles, historico em member_transfers
- ✓ Gerar PDF da ficha → dados completos, foto incluída

FASE 3 Secretaria — Documentos Eclesiásticos

Dep: Membros

1 semana

document_types

ministry_id • nome • template_html text • variaveis_disponiveis jsonb •
requerassinatura bool • ativo bool

Template com variáveis: {{nome}}, {{cpf}}, {{batismo}}, {{cargo}}, {{unidade}}, {{data_emissao}}, etc.

documents

ministry_id • party_id FK • unit_id FK • type_id FK • emitido_por uuid •
data_emissao date • numero_protocolo text • arquivo_url text • observacoes text

- › **Tipos de documentos configuráveis:** Declaração de Membro Ativo, Certidão de Batismo, Carta de Transferência, Ofício ao Ministério, Declaração de Cargo
- › **Editor de template HTML** por ministério — cada campo pode customizar o texto e layout
- › **Geração automática de PDF:** variáveis do membro preenchidas automaticamente via Puppeteer ou jsPDF
- › **Número de protocolo automático** sequencial por ministério/ano
- › **Histórico de documentos por membro:** quem emitiu, quando, qual documento
- › **Assinatura digital futura:** campo requerassinatura prepara o fluxo

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Emitir Declaração de Membro Ativo → PDF com dados do membro, logo da igreja, data e número de protocolo
- ✓ Carta de Transferência → PDF enviado para o membro e registrado no histórico

FASE 4 EBD — Escola Bíblica Dominical

Dep: Membros + Unidades

1 semana

ebd_classes

ministry_id • unit_id FK • nome • faixa_etaria • professor_party_id FK •
dia_semana • horario • ativo

ebd_roll_calls

class_id FK • ministry_id • data date • total_presentes int • visitantes int •
criado_por uuid

ebd_attendances

roll_call_id FK • ministry_id • party_id FK • presente bool • justificativa text

- › Turmas por unidade com professor responsável e faixa etária configurável
- › Chamada semanal rápida: lista de alunos com toggle presente/ausente, campo visitantes avulsos
- › Alerta automático para alunos com mais de 2 faltas consecutivas
- › Relatório de frequência: por turma, período, por aluno — exportável em PDF/XLSX
- › Dashboard EBD: total de alunos, média de frequência do mês, aniversariantes da semana
- › N3 vê apenas turmas da sua unidade e filhas

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Criar turma, adicionar alunos (Parties MEMBRO) e realizar chamada → presença registrada
- ✓ Gerar relatório de frequência do mês → percentual por aluno calculado corretamente
- ✓ Aluno com 3 faltas → aparece no painel de atenção pastoral

school_semesters

ministry_id · nome · data_inicio · data_fim · ativo

school_disciplines

semester_id FK · ministry_id · nome · carga_horaria int · professor_party_id FK ·
nota_minima numeric · frequencia_minima numeric

school_enrollments / school_grades / school_attendance

Matrícula por disciplina · Notas (AP1, AP2, FINAL, RECUPERAÇÃO) · Presença por
aula

- › Semestres com datas de início/fim e ativação/encerramento manual
- › Disciplinas com professor, carga horária, nota mínima (geralmente 6.0) e frequência mínima (geralmente 75%)
- › Matrícula de Party com role ALUNO — um membro se torna aluno ao se matricular
- › Lançamento de presença por aula e notas por avaliação (AP1, AP2, Final, Recuperação)
- › Cálculo automático de média e situação (APROVADO, REPROVADO_NOTA, REPROVADO_FREQUÊNCIA)
- › Boletim do aluno em PDF: disciplinas, médias, situação, % de presença
- › Relatório de formandos por semestre
- › Histórico acadêmico completo por aluno (todos os semestres)

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Criar semestre, adicionar 2 disciplinas, matricular 3 alunos → lançar notas → médias calculadas corretamente
- ✓ Aluno com frequência abaixo de 75% → situação REPROVADO_FREQUÊNCIA mesmo com nota aprovada
- ✓ Gerar boletim PDF → notas e situação corretas

FASE 6

Cursos Livres

Dep: Membros

1 semana

`courses`

`ministry_id` · `titulo` · `descricao` · `categoria` · `carga_horaria` · `data_inicio` ·
`vagas` · `publico_alvo` · `ativo`

`course_enrollments` / `course_certificates`

Inscrições com status (INSCRITO, CONCLUÍDO, DESISTÊNCIA) · Certificados com data de emissão e URL do PDF

- › Catálogo de cursos livres com categoria, carga horária e controle de vagas
- › Inscrições de membros ou externos (externos via Party sem role MEMBRO)
- › Controle de presença por encontro/aula do curso
- › Conclusão automática quando todas as aulas marcadas como presentes
- › Emissão de certificado em PDF com template customizável por ministério
- › Relatório de cursos: inscritos, concluídos, desistências por curso

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Criar curso, inscrever participante, registrar conclusão → certificado gerado em PDF com dados corretos

FASE 7

Financeiro — Fund Accounting Eclesiástico

Dep: Membros + Unidades

3-4 semanas

CRÍTICO

Módulo mais complexo tecnicamente. Requer plano de contas hierárquico, múltiplas contas por unidade, Fund Accounting (separação de receitas por finalidade) e relatórios contábeis exigidos pela legislação para entidades religiosas sem fins lucrativos.



FUND ACCOUNTING — REQUISITO ENTERPRISE

Igrejas devem separar dízimos, ofertas gerais, missões, construção, eventos especiais em fundos distintos. Um único lançamento não pode misturar fundos. O plano de contas deve suportar no mínimo 3 níveis hierárquicos (Grupo → Conta → Subconta).

`fin_accounts`

`ministry_id` · `unit_id` FK · `nome` · `tipo` (BANCO, CAIXA, POUPANÇA, INVESTIMENTO) ·
`banco` · `agencia` · `conta` · `saldo_inicial` · `ativo`

`fin_categories` (Plano de Contas)

`ministry_id` · `parent_id` (hierarquia) · `codigo` · `nome` · `tipo` (RECEITA, DESPESA) ·
`fundo` (DIZIMO, OFERTA, MISSOES, CONSTRUCAO, EVENTO, OUTRO)

`fin_transactions`

`ministry_id` · `unit_id` FK · `account_id` FK · `category_id` FK · `party_id` FK (opcional
– dízimo do membro) · `tipo` (ENTRADA, SAÍDA) · `valor` numeric(12,2) · `data` date ·
`descricao` · `comprovante_url` · `aprovado_por` uuid · `status` (PENDENTE, APROVADO,
REJEITADO)

`fin_budgets`

`ministry_id` · `unit_id` · `category_id` · `ano` int · `mes` int · `valor_previsto`
numeric(12,2)

- › **Plano de Contas hierárquico** configurável por ministério: código, nome, tipo (RECEITA/DESPESA), fundo
- › **Contas bancárias e caixas** por unidade: saldo em tempo real calculado por lançamentos
- › **Lançamentos com party_id**: dízimos e ofertas vinculados ao membro — histórico de contribuição por pessoa
- › **Fluxo de aprovação**: despesas acima de valor configurável requerem aprovação de N2 (Admin Sede)
- › **Conciliação bancária**: comparativo extrato importado vs. lançamentos no sistema
- › **Fluxo de caixa**: mensal e anual por unidade, consolidado por campo
- › **Orçamento vs. realizado**: comparativo por categoria e mês

- › **Relatórios exportáveis em PDF:** DRE simplificado, Balancete mensal, extrato de contribuições por membro
- › **Conformidade fiscal:** relatório anual de doações para declaração de IR (exportação em XLSX por CPF)

Regras de Negócio Financeiro

- N4 (Usuário Local) só lança receitas da sua própria unidade — sem acesso a despesas
- N3 (Admin Setor) vê e lança para sua unidade e filhas — não vê outros setores
- N2 (Admin Sede) vê tudo do ministério e aprova despesas acima do limite
- Lançamentos aprovados não podem ser deletados — apenas estornados (novo lançamento de sinal contrário)
- Saldo de conta nunca pode ficar negativo sem flag de autorização explícita de N1

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Lançar dízimo vinculado a membro → saldo da conta atualizado, histórico de contribuição do membro registrado
- ✓ Criar despesa acima do limite → status PENDENTE até aprovação de N2
- ✓ Gerar fluxo de caixa de janeiro → todas as entradas e saídas listadas com saldo final
- ✓ Exportar relatório anual de contribuições → planilha com CPF e totais por contribuinte
- ✓ Login como N3 Setor 01 → não ver lançamentos do Setor 02

FASE 8 Eventos e Agenda

Dep: Membros + Unidades

1 semana

events

ministry_id · unit_id FK · titulo · tipo (CULTO, CONFERÊNCIA, RETIRO, REUNIÃO, EBD_ESPECIAL, OUTRO) · descricao · data_inicio · data_fim · local · capacidade
int · publico bool · ativo

event_registrations / event_checkins

Inscrições com status (INSCRITO, CONFIRMADO, LISTA_ESPERA, CANCELADO) · Check-in com timestamp e método (MANUAL, QR_CODE)

- › Calendário com visualização mensal, semanal e em lista — filtro por tipo e unidade

- › Eventos públicos (visíveis para todos) e privados (apenas para usuários do sistema)
- › Inscrições com controle de vagas e lista de espera automática
- › Check-in manual ou por QR Code (gerado para cada inscrito)
- › Relatório de participação: total inscritos, presentes, ausentes por evento
- › Integração futura com Comunicação (Fase 11): lembrete automático por WhatsApp/e-mail antes do evento

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Criar retiro com limite de 50 vagas → 51ª inscrição vai para lista de espera
- ✓ Check-in via QR Code → registrado com timestamp
- ✓ Gerar lista de presença → participantes confirmados listados

FASE 9 Patrimônio e Facilities

Dep: Unidades

1 semana

`asset_categories / assets / asset_movements`

Categorias com taxa de depreciação anual · Bens com: nome, número de série, valor de aquisição, data de aquisição, fornecedor, estado (BOM, REGULAR, RUIM), foto_url, nota_fiscal_url
Movimentações: tipo (COMPRA, TRANSFERÊNCIA, BAIXA, MANUTENÇÃO), unit_destino, responsável, data

- › Inventário completo de bens por unidade com foto, nota fiscal, número de série
- › Categorias configuráveis com taxa de depreciação anual automática
- › Cálculo automático de valor depreciado atual ($\text{valor_aquisição} \times (1 - \text{taxa})^{\text{anos}}$)
- › Movimentação de bem entre unidades com histórico completo e responsável
- › Baixa de bem (descarte, roubo, perda) com justificativa
- › Ordens de serviço de manutenção vinculadas ao bem
- › Relatório patrimonial: valor total por unidade, valor depreciado, bens para manutenção

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Cadastrar bem (ex: projetor R\$5.000, 5 anos de vida útil) → valor depreciado calculado corretamente após 2 anos
- ✓ Transferir bem entre igrejas → histórico de movimentação registrado

✓ Relatório patrimonial consolidado → totais corretos por unidade

FASE 10 Ocorrências Pastorais

Dep: Membros + N2+

1 semana

SENSÍVEL

Módulo de alta sensibilidade. Ocorrências pastorais (disciplina, conflito, situação pessoal) são visíveis apenas para N2 (Admin Sede) e acima. RLS deve ser validada com rigor neste módulo — um N4 nunca pode visualizar, mesmo via SQL direto.

`occurrence_types / occurrences / occurrence_comments`

Tipos: nome, severidade (BAIXA, MÉDIA, ALTA, CRÍTICA), nível_mínimo_visibilidade (2, 3 ou 4)

Ocorrência: party_id, unit_id, tipo, descricao, status (ABERTA, EM_ACOMPANHAMENTO, RESOLVIDA, ARQUIVADA), responsavel

Comentários internos com autor, data, editável apenas pelo autor em até 24h

- › Tipos de ocorrência configuráveis com severidade e nível mínimo de visibilidade
- › Ocorrências CRÍTICAS: notificação automática para N1 (Admin Campo)
- › Fluxo de status: ABERTA → EM_ACOMPANHAMENTO → RESOLVIDA → ARQUIVADA
- › Comentários internos com histórico de acompanhamento pastoral
- › Relatório gerencial por tipo, período e unidade (apenas N2+)
- › Anonimização após 5 anos (nome do membro substituído por hash) — LGPD

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Registrar ocorrência ALTA como N2 → visível para N1 e N2, invisível para N3 e N4
- ✓ Login como N4 → /dashboard/ocorrencias redireciona (403), e query direta retorna zero rows (RLS)
- ✓ Resolver ocorrência → status atualizado, comentário de resolução registrado

FASE 11 Comunicação Omnichannel

Dep: Membros + Eventos

2–3 semanas

ENTERPRISE

`comm_templates / comm_campaigns / comm_logs`

Templates: canal (EMAIL, WHATSAPP, PUSH), assunto, corpo_html, variaveis_disponiveis

Campanhas: segmento (unidade, nível, filtro livre), agendamento, status (RASCUNHO, AGENDADA, ENVIANDO, CONCLUÍDA)

Logs: destinatário, canal, status (ENVIADO, LIDO, FALHA), timestamp

- › **E-mail:** integração SendGrid — campanhas segmentadas por unidade, status, faixa etária, etc.
- › **WhatsApp Business API:** integração Z-API ou Twilio — mensagens individuais (convite, aniversário) e em massa (eventos, avisos)
- › **Push Notifications:** via web (navegador) usando Web Push API — anúncios urgentes
- › **Automações:** aniversário do membro, ausência de 3 domingos, confirmação de inscrição em evento, lembrete 24h antes do evento
- › **Workers assíncronos:** Supabase Edge Functions com queue para envio em lote sem bloquear o servidor
- › **Funil de visitantes:** sequência de follow-up automática pós-primeira visita (D+1, D+7, D+30)
- › Relatório de entrega: taxa de abertura de e-mail, entrega de WhatsApp, cliques

✓ CRITÉRIOS DE VALIDAÇÃO

- ✓ Criar campanha para todos os membros do Setor 01 → WhatsApp enviado apenas para Setor 01
- ✓ Membro aniversariante → mensagem automática enviada na data
- ✓ Log de envio → registros de ENVIADO/FALHA por destinatário

FASE 12

Voluntários & Escalas de Serviço

Dep: Membros + Eventos

2 semanas

ENTERPRISE

`volunteer_profiles / ministry_teams / service_schedules / schedule_confirmations`

Perfil: habilidades, disponibilidade semanal, ministério(s), observações

Times: louvor, recepção, mídia, infantil, limpeza, etc. — por unidade

Escala: data, time_id, lista de escalados, status (PENDENTE_CONFIRMAÇÃO,

CONFIRMADA)

Confirmação: voluntário, resposta (CONFIRMADO, IMPOSSIBILITADO, motivo)

- › Cadastro de voluntários com habilidades, disponibilidade e ministérios de atuação
- › Times de ministério por unidade: louvor, mídia, recepção, limpeza, infantil, etc.
- › Geração de escala manual ou semi-automática por disponibilidade e rodízio
- › Confirmação de escala via WhatsApp (integração com Fase 11) ou link no sistema
- › Substituição: voluntário indisponível sugere substituto ou líder escolhe
- › Histórico de serviços por voluntário
- › Relatório de presença nas escalas por período

FASE 13

Ministério Infantil — Check-in Seguro

Dep: Membros + Voluntários

2 semanas

ENTERPRISE

[child_profiles](#) / [child_responsibles](#) / [child_classes](#) / [child_checkins](#)

Perfil da criança: nome, nascimento, alergias, necessidades especiais, foto, observações médicas

Responsáveis: vinculação child → party (pai, mãe, responsável legal) com permissão de retirada

Turma: faixa etária, professor, capacidade

Check-in: criança, turma, responsável_entrada, responsável_saida, código_único, timestamp

- › Check-in de criança: responsável apresenta código (QR ou PIN de 4 dígitos) na entrada
- › Etiqueta de identificação impressa automaticamente com nome, turma e código de segurança
- › Vinculação responsável ↔ criança: apenas responsáveis cadastrados podem retirar
- › Check-out: responsável apresenta o mesmo código para retirar a criança
- › Push notification para responsável (Fase 11) se criança for chamada (emergência)
- › Controle de alergias e necessidades especiais visível para professor da turma
- › Relatório de frequência infantil por turma e período

FASE 14

Governança, Compliance e LGPD

Dep: Fase 1 + Membros

3 semanas

ENTERPRISE

`governance_meetings / governance_minutes / mandates / votes / lgpd_consentments / audit_logs`

Reuniões: tipo (ASSEMBLEIA, CONSELHO, PLENÁRIA), pauta, data, participantes

Atas: conteúdo, assinaturas digitais, arquivo PDF

Mandatos: cargo, party_id, data_inicio, data_fim, situação

Votos: pauta, opções, contagem, resultado

Consentimentos LGPD: finalidade, data, IP, versão do termo

Audit Trail: tabela, ação (INSERT/UPDATE/DELETE), dados_antes, dados_depois, user_id, ip, timestamp

- › **Atas de reunião:** editor de texto, participantes, assinaturas digitais, arquivo gerado em PDF
- › **Controle de mandatos:** pastores, diáconos, presbíteros — início, fim, renovação, histórico
- › **Votações eletrônicas:** criação de pauta, votação por membros elegíveis, contagem automática e resultado
- › **Audit Trail completo:** toda ação no sistema registrada com timestamp, IP, usuário e dados antes/depois
- › **LGPD:** consentimento de uso de dados na criação de Party, exportação de dados do membro (direito de portabilidade), solicitação de exclusão (anonimização)
- › **MFA (2FA):** autenticação de dois fatores para N0 e N1 obrigatória
- › **SSO:** integração opcional com Google Workspace (OAuth2) para ministérios que utilizam Google

FASE 15

Analytics, BI e Dashboards Executivos

Dep: Todos os módulos

2–3 semanas

ENTERPRISE

- › **Dashboard executivo:** KPIs em tempo real — total de membros, crescimento batismal, taxa de retenção de visitantes, evasão, frequência EBD, arrecadação mensal
- › **Comparativo entre unidades:** benchmarking Setor 01 vs. Setor 02 vs. Piracicaba vs. São Paulo

- › **Análise de tendências:** gráficos de crescimento mensal, sazonalidade financeira, pico de eventos
- › **IA para insights automáticos:** "Setor 01 teve queda de 12% na frequência jovem nos últimos 60 dias" — via Supabase Edge Function com OpenAI API
- › **Relatórios agendados:** envio automático semanal/mensal para liderança via e-mail (PDF ou link)
- › **Views materializadas:** atualizadas periodicamente via pg_cron para não sobrecarregar queries analíticas
- › **Exportação:** XLSX, CSV, integração com Power BI / Google Looker Studio via API

FASE 16

API Pública REST e Integrações

Dep: Financeiro + Membros

2 semanas

ENTERPRISE

`api_clients` / `api_keys` / `webhook_endpoints` / `webhook_logs`

Cientes: nome, ministry_id, escopos permitidos, rate_limit

Chaves: hash SHA-256, expires_at, last_used_at

Webhooks: url, eventos assinados, secret_hmac, ativo

Logs: evento, payload, status_http, tentativas, proximo_retry

- › **API REST pública:** endpoints de leitura para membros, financeiro e eventos — autenticação via API Key
- › **Documentação OpenAPI:** gerada automaticamente com Swagger UI em /api/docs
- › **Rate limiting:** por API Key no middleware.ts (100 req/min padrão)
- › **Webhooks:** eventos em tempo real (membro_criado, transacao_criada, evento_publicado) para sistemas externos
- › **Integrações pré-construídas:** Conta Azul (contabilidade), PagSeguro/Asaas (pagamentos), Mailchimp (e-mail marketing)
- › **Sandbox:** ambiente de homologação com dados fictícios para integrar sem afetar produção

FASE 17

PWA e App Mobile Nativo

Dep: Fases 1-10 estáveis

4-6 semanas

FUTURO

- › **PWA (Progressive Web App):** Next.js já suporta — manifesto, service worker, instalação no homescreen, modo offline básico (rotas em cache)
- › **Push Notifications Web:** Web Push API para notificações nativas no navegador mobile
- › **App Nativo (React Native + Expo):** para funcionalidades que exigem hardware — câmera para check-in infantil via QR Code, modo totalmente offline, notificações push nativas iOS/Android
- › **Compartilhamento de lógica:** TypeScript + zod schemas compartilhados entre web e mobile via monorepo (Turborepo)
- › **White-label:** assets (ícone, splash, nome) configuráveis por ministério via arquivo de config — um único codebase gera N apps diferentes

6. Regras de Governança do Processo de Desenvolvimento



GOVERNANÇA — APLICA-SE A TODOS OS PROJETOS DESTA ORGANIZAÇÃO

As regras abaixo não são recomendações. São restrições de processo vinculantes para qualquer sistema desenvolvido por esta equipe, independente de cliente, prazo ou pressão externa.

REGRA #1

Nenhum módulo começa antes do anterior ser totalmente validado

Um módulo está **validado** quando passou pelo ciclo completo de testes documentado abaixo — não apenas quando a interface funciona visualmente. Avançar antes da validação é proibido mesmo sob pressão de prazo.

O QUE SIGNIFICA "TOTALMENTE VALIDADO"

- 01 CRUD completo testado** — criar, editar, listar, filtrar, inativar/reactivar todos funcionando sem erros
- 02 Isolamento multi-tenant** — dado de Campo A não aparece para Campo B em nenhum cenário
- 03 RBAC por nível testado** — testado com N0, N1, N3 e N4. Cada nível vê exatamente o que deve ver, nada a mais
- 04 RLS verificado no banco** — query direta no Supabase Dashboard com role de N4 retorna zero rows nas tabelas restritas
- 05 Relatórios e PDFs** — quando o módulo os gera, verificar dados corretos e layout adequado
- 06 Critério de validação da fase aprovado** — o cenário específico descrito neste Roadmap para a fase foi executado com sucesso

POR QUE ESTA REGRA EXISTE

Cada módulo neste sistema depende diretamente do anterior. Avançar com fundação inconsistente multiplica o retrabalho em cascata. Uma falha no Party Pattern (Fase 2) descoberta durante a Fase 5 significa reescrever 3 módulos. Uma falha de RLS descoberta em produção é uma violação de dados de todos os ministérios.

REGRA #2 Migrations são imutáveis após execução

Uma migration executada no banco **nunca** é editada. Toda correção ou ajuste no schema gera uma nova migration com o próximo número sequencial. Isso garante rastreabilidade completa do histórico do banco em qualquer ambiente.

REGRA #3 RLS é implementado antes de qualquer dado operacional

Toda nova tabela tem RLS ativado e políticas definidas na própria migration de criação, nunca retroativamente. Não existe "implementar a segurança depois" — dado sem RLS é dado exposto.

REGRA #4 Este Roadmap é o documento de referência — não memória ou conversa

Toda decisão arquitetural, toda mudança de escopo, todo novo módulo ou regra de negócio deve ser registrada neste Roadmap antes de ser codificada. O documento é atualizado a cada fase concluída e a cada decisão relevante tomada.

7. Regras Gerais de Desenvolvimento

Banco de Dados e Segurança

● REGRA ABSOLUTA — MULTI-TENANT

Todo INSERT deve incluir `ministry_id` explícito. Todo SELECT deve filtrar por `ministry_id`. Toda nova tabela deve ter RLS ativo ANTES de qualquer dado ser inserido. Jamais omitir o isolamento multi-tenant em nenhuma camada.

● REGRA ABSOLUTA — OCORRÊNCIAS E MÓDULOS SENSÍVEIS

RLS por nível (level) para tabelas sensíveis — não apenas por `ministry_id`. Testar SEMPRE com login de N4 e confirmar que rows retornam vazios via query direta no Supabase Dashboard.

⚠ NOMENCLATURA DE MIGRATIONS

Sequencial: 005_nome_descritivo.sql. Nunca reeditar migration já executada — criar nova migration de correção.

- › Toda nova tabela deve ter: `id uuid PK DEFAULT gen_random_uuid()`, `ministry_id uuid NOT NULL`, `created_at timestamptz DEFAULT now()`, `updated_at timestamptz`
- › Trigger automático de `updated_at` em todas as tabelas que têm este campo
- › Usar `.maybeSingle()` em vez de `.single()` para queries que podem retornar null
- › Nunca fazer `SELECT *` — sempre listar colunas necessárias (performance e segurança)
- › Índices obrigatórios: `(ministry_id)`, `(ministry_id, unit_id)`, `(ministry_id, party_id)` nas tabelas que referenciam esses campos

Frontend — Next.js 15 App Router

● REGRA ABSOLUTA — SERVER VS. CLIENT COMPONENTS

Event handlers (`onMouseEnter`, `onClick`, `onChange`, etc.) nunca em Server Components. Extrair para Client Components com "use client" em arquivo separado quando necessário.

⚠ SERVER ACTIONS

Server Actions são passadas como props bound para Client Components — nunca importadas diretamente dentro de um Client Component (causa erro Turbopack HMR "module factory not available").

- › Server Components para fetch de dados (cookies(), createClient(), queries)
- › Client Components (useState, useTransition, useRef) para toda interatividade
- › revalidatePath() em toda Server Action de mutação — sem stale data
- › Cookies modificados apenas em Server Actions ou Route Handlers — nunca em render de Server Component
- › Usar useTransition + isPending para todos os forms com Server Action — UX de loading sem travar

Processo de Validação por Módulo

FLUXO OBRIGATÓRIO ANTES DE AVANÇAR

Cada fase deve passar pelo checklist completo abaixo antes de iniciar a fase seguinte. Não há exceções.

CHECKLIST DE VALIDAÇÃO POR MÓDULO

- ☐ CRUD completo: criar, editar, listar, filtrar, inativar
- ☐ Testar como N0 Super Master (deve ver todos os campos)
- ☐ Testar como N1 Admin Campo (deve ver apenas o seu campo)
- ☐ Testar como N3 Admin Setor (deve ver apenas sua unidade e filhas)
- ☐ Testar como N4 Usuário Local (acesso mínimo — apenas sua Igreja)
- ☐ Confirmar isolamento: dado de Piracicaba NÃO aparece para SP e vice-versa
- ☐ Verificar RLS via query direta no Supabase Dashboard com role de usuário
- ☐ Gerar relatório (quando aplicável) e verificar dados corretos
- ☐ Gerar PDF (quando aplicável) e verificar layout

Infraestrutura e Deploy (Para Produção)

- › **vercel.json** — configurar headers de segurança: CSP, X-Frame-Options, HSTS, X-Content-Type-Options
- › **.github/workflows/ci.yml** — pipeline de lint (ESLint) + build verification a cada push
- › **next.config.ts** — Content Security Policy via headers(), imagens de domínios autorizados
- › **Rate limiting no middleware.ts** — proteger /api/* e /auth/* de brute-force

- › **Sentry** — monitoramento de erros em produção (sentry.client.config.ts + sentry.server.config.ts)
- › **public/manifest.json** — para PWA fase 17: nome, ícones, cores, orientação

Estimativa de Custo de Infraestrutura (Produção)

Supabase Pro	~\$25/mês — banco, auth, storage 100GB
Vercel Pro	~\$20/mês — frontend, Edge Functions
SendGrid (e-mail)	~\$15–50/mês conforme volume
WhatsApp API (Z-API)	~\$50–200/mês conforme volume de mensagens
Sentry	~\$0 (gratuito até 5k errors/mês)
Total Infra	~\$110–300/mês para operação média
Esforço total (17 fases)	Estimativa: 26–38 meses (team 2–3 devs) · 8–10 meses (Fases 1–10 em foco)

→ Próxima Ação: FASE 1C — Configuração de Módulos

Última entrega da Fase 1 antes de iniciar o NÚCLEO do sistema (Fase 2 — Membros)

- Criar página /dashboard/admin/modulos com lista dos 9 módulos e toggle por ministério
- Server Action de toggle: atualiza ministry_modules.is_active + revalida contexto
- Exibir nome, ícone e descrição de cada módulo para orientar o admin
- Validar: desativar Financeiro no SP → item some do sidebar após novo login do usuário SP
- Após 1C validada → iniciar planejamento da Fase 2 (Membros) com foco em Party Pattern e migration 005